



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO PIAUÍ

Campus Piripiri

Tecnólogo em Análise e Desenvolvimento de Sistemas

Francisco Igor Silva Santos

Sistema para Emissão, Gestão e Autenticação de Certificados

Piripiri – PI

2025

Francisco Igor Silva Santos

2024116TADS0030

Sistema para Emissão, Gestão e Autenticação de Certificados

Relatório Técnico de Software apresentado ao curso de Tecnólogo em Análise e Desenvolvimento de Sistemas, como requisito parcial para a conclusão do Trabalho de Conclusão de Curso (TCC).

Orientador(a): Prof. Mayllon Veras da Silva

Piripiri – PI

2025

Resumo

Texto resumido do projeto, objetivos, tecnologias e resultados. Palavras-chave: software, desenvolvimento, tecnologia.

Abstract

Short text in English summarizing the work. Keywords: software, development, technology.

Sumário

1.0 Introdução	6
1.1 Objetivos	7
1.1.1 Geral	7
1.1.2 Específicos	7
2.0 Tecnologias Envolvidas	8
2.1 Frontend	8
2.1.1 React 19	8
2.1.2 TypeScript	9
2.1.3 Tailwind CSS	9
2.1.4 shadcn/ui	10
2.1.5 TanStack Router	10
2.1.6 TanStack React Query	11
2.1.7 React Hook Form	11
2.1.8 Sonner	11
2.1.9 Recharts	12
2.1.10 date-fns	12
2.1.11 Vite	13
2.2 Integração com a API REST	13
2.2.1 Zod	13
2.3 Backend	14
2.3.1 Next.js 16	14
2.3.2 Supabase e PostgreSQL	15
2.3.3 PDFKit	16
2.3.4 Zod	16
2.3.5 Arquitetura e Padrões	17
2.4 Infraestrutura e Ferramentas	17
2.4.1 Render	17
2.4.2 Jest	18
2.4.3 ESLint e Prettier	18
2.4.4 Git e GitHub	18

2.4.5	Visual Studio Code	19
2.5	Síntese das Tecnologias	19
3.0	Modelagem do Projeto	21
3.1	Levantamento de Requisitos	21
3.1.1	Módulo de Autenticação e Controle de Acesso	21
3.1.2	Módulo de Templates de Certificados	21
3.1.3	Módulo de Entrada de Dados	22
3.1.4	Módulo de Geração de Certificados	22
3.1.5	Módulo de Envio e Comunicação	22
3.1.6	Módulo de Validação Pública e Antifraude	23
3.1.7	Módulo de Assinaturas Digitais	23
3.1.8	Módulo de Dashboard e Gestão	23
3.1.9	Módulo de Auditoria e Segurança Operacional	23
3.1.10	Módulo de Gestão de Cursos/Eventos	24
3.1.11	Módulo de Participantes	24
3.1.12	Módulo Multiempresa / Multi-tenant	24
3.1.13	Módulo de Configurações do Sistema	25
3.1.14	Módulo de Armazenamento e Arquivos	25
3.1.15	Módulo de Notificações e Alertas	25
3.1.16	Módulo de Monitoramento e Logs	25
3.1.17	Módulo de Suporte, Operação e Manutenção	25
3.1.18	Segurança	26
3.1.19	Desempenho e Escalabilidade	26
3.1.20	Disponibilidade e Confiabilidade	27
3.1.21	Usabilidade e Acessibilidade	27
3.1.22	Compatibilidade e Integração	27
3.1.23	Armazenamento e Retenção	28
3.1.24	Conformidade Legal e LGPD	28
3.1.25	Arquitetura e Tecnologia	28
3.1.26	Manutenção e Monitoramento	29
3.1.27	Internacionalização	29
3.2	Diagramas de casos de uso	30

3.3	Diagramas de classe	30
3.4	Arquitetura do Sistema	31
3.5	Diagrama Entidade-Relacionamento	33
3.6	Interfaces	33
4.0	Software	33
4.1	Implantação	33
4.2	Testes	33
5.0	Considerações Finais	34
	Referências	36
6.0	Anexos	37
6.1	Declaração de Entrega do Código-Fonte	37
6.2	Declaração de Submissão ao NIT	37

1.0 Introdução

A emissão de certificados é uma atividade essencial para empresas de treinamentos, consultorias e instituições educacionais que promovem cursos, capacitações e eventos. Apesar da relevância desse processo, grande parte das organizações ainda depende de procedimentos manuais ou soluções improvisadas, utilizando ferramentas como Google Planilhas, Google Docs, PowerPoint e complementos externos.

Essas práticas resultam em retrabalho, inconsistências, falta de padronização e dificuldade de controle operacional. Além disso, ferramentas como AutoCrat e documentos espalhados entre diferentes plataformas não foram projetados para uso profissional em escala, o que gera limitação, fragilidade e risco de inconsistências.

A ausência de um sistema integrado compromete a rastreabilidade e a credibilidade dos certificados, uma vez que não há mecanismos oficiais de validação pública, histórico centralizado, indicadores gerenciais ou emissão em lote eficiente. Estudos recentes mostram que sistemas baseados em QR Code têm se destacado como alternativa eficaz para verificação de credenciais acadêmicas [1], combinando criptografia e códigos bidimensionais para garantir a autenticidade dos documentos [2]. Esse cenário se agrava com o crescimento de cursos online e da necessidade de certificação confiável.

Diante disso, justifica-se o desenvolvimento de uma solução centralizada e automatizada, capaz de reduzir erros humanos, otimizar o tempo operacional e garantir segurança, conformidade e profissionalismo. Recursos como códigos únicos, validação via QR Code e assinaturas digitais ampliam a confiabilidade dos certificados e fortalecem a imagem institucional das organizações emissoras [3]. Soluções semelhantes já foram implementadas com sucesso utilizando criptografia AES e QR Code para verificação de certificados acadêmicos [4].

O presente trabalho consiste no desenvolvimento de um Sistema Web — produto do tipo SaaS (Software as a Service) com arquitetura multi-tenant — destinado à emissão, gestão e autenticação de certificados. O sistema foi projetado com base nas necessidades reais de uma empresa parceira do ramo de treinamentos corporativos, que atualmente enfrenta as limitações operacionais descritas. Dessa forma, o escopo abrange desde a automatização da criação e personalização de certificados até a disponibilização de um portal público de validação, visando substituir processos manuais por uma plataforma profissional, segura e escalável.

Este relatório técnico está organizado da seguinte forma: a Seção 2 apresenta as tecnologias utilizadas no desenvolvimento; a Seção 3 descreve a modelagem do sistema, incluindo levanta-

mento de requisitos, diagramas de casos de uso, classes, arquitetura e entidade-relacionamento; a Seção 4 detalha a implementação do software; e a Seção 5 traz as considerações finais.

1.1 Objetivos

1.1.1 Geral

Projetar e implementar um sistema web automatizado para emissão, gestão, envio e validação de certificados, destinado a empresas de treinamento corporativo, instituições educacionais e demais organizações que necessitem certificar participantes em cursos, capacitações e eventos, a fim de eliminar a dependência de processos manuais e soluções improvisadas, garantindo padronização dos documentos, rastreabilidade do histórico de emissões, autenticação confiável via código único e QR Code, emissão em lote eficiente e centralização dos dados em uma única plataforma.

1.1.2 Específicos

- implementar um módulo de templates configuráveis para a criação de certificados;
- desenvolver a funcionalidade de importação de dados a partir de arquivos CSV e Excel (.xlsx) com processamento em fila para emissão em lote;
- implementar um módulo de notificação por e-mail com suporte a variáveis dinâmicas e reenvio automático em caso de falha;
- desenvolver um mecanismo de autenticação pública via código único e QR Code para garantir a integridade dos certificados e prevenir fraudes;
- desenvolver um módulo de dashboard com indicadores gerenciais, filtros por período, curso e status;
- planejar e executar testes unitários e de integração para validação dos módulos implementados;
- configurar o ambiente de implantação do sistema em infraestrutura cloud utilizando containerização.

2.0 Tecnologias Envolvidas

Este capítulo apresenta as tecnologias empregadas no desenvolvimento do sistema, organizadas em frontend, backend e infraestrutura, com uma discussão sobre o papel de cada uma, os motivos de sua escolha e as alternativas consideradas durante o processo de decisão.

Arquiteturalmente, o sistema é composto por duas aplicações independentes, cada uma com seu próprio repositório, dependências, ferramenta de build e pipeline de deploy: (1) o frontend, uma *Single Page Application* (SPA) construída com React 19 e Vite, que utiliza o TanStack Router para roteamento no lado do cliente e se comunica com o backend via requisições HTTP; e (2) o backend, uma API REST que utiliza o Next.js exclusivamente como framework de servidor — seus *Route Handlers* processam as requisições, mas o Next.js não renderiza páginas nem serve conteúdo frontend. As seções a seguir detalham as tecnologias de cada camada, deixando explícito o escopo de atuação de cada uma nessa arquitetura de duas aplicações.

2.1 Frontend

A camada de apresentação foi construída com tecnologias modernas do ecossistema JavaScript, priorizando tipagem estática, experiência de desenvolvimento produtiva e performance em tempo de execução.

2.1.1 React 19

React é uma biblioteca JavaScript para construção de interfaces de usuário baseada no conceito de componentes reutilizáveis e estado unidirecional [5]. A versão 19 introduz melhorias significativas no modelo de concorrência, como o novo hook `use()`, que permite leitura assíncrona de recursos diretamente no corpo do componente, e as *Actions* para gerenciamento nativo de formulários, reduzindo a necessidade de bibliotecas externas para controle de estado de formulários.

A escolha do React 19 baseou-se em dois fatores principais: (1) maturidade do ecossistema, com ampla disponibilidade de bibliotecas e documentação; e (2) modelo de componentes que favorece a modularidade e a testabilidade do código. Alternativas como Vue.js e Angular foram descartadas respectivamente pelo ecossistema menos consolidado para o tipo de aplicação e pela curva de aprendizado mais steep.

2.1.2 TypeScript

TypeScript é um superconjunto tipado de JavaScript que compila para JavaScript puro. Seu sistema de tipos estrutural permite detectar erros comuns em tempo de compilação, como acesso a propriedades inexistentes, passagem de argumentos com tipos incompatíveis e retorno incorreto de funções [6].

A adoção do TypeScript em vez de JavaScript puro deveu-se à necessidade de contratos explícitos entre módulos e à segurança proporcionada pela verificação estática de tipos: erros como acesso a propriedades inexistentes ou argumentos com tipos incorretos são detectados em tempo de compilação, e não em runtime. Alternativas como Flow (Facebook) foram descartadas por seu ecossistema significativamente menor — o Flow não é suportado nativamente por ferramentas como ESLint, Prettier e o próprio Vite — e pela comunidade majoritariamente concentrada no TypeScript, que resulta em melhor documentação, mais tipos publicados (@types/*) e suporte de primeira classe em IDEs.

No projeto, o TypeScript foi utilizado em toda a camada frontend. O recurso de *generics* possibilitou a criação de hooks e utilitários reutilizáveis com segurança de tipos, enquanto *union types* e *discriminated unions* foram empregados para modelar estados de carregamento, erro e sucesso das requisições assíncronas.

2.1.3 Tailwind CSS

Tailwind CSS é um framework de estilização baseado no paradigma *utility-first*, fornecendo classes atômicas de baixo nível (como `flex`, `p-4`, `text-lg`) que são compostas diretamente no markup HTML/JSX [7]. Diferentemente de abordagens tradicionais como Bootstrap (baseada em componentes pré-construídos), o Tailwind oferece maior flexibilidade de design sem a necessidade de sobrescrever estilos padrão.

Na versão 4, o Tailwind adota uma abordagem *CSS-first*, eliminando a necessidade do arquivo `tailwind.config.js`. A configuração é feita diretamente via CSS com o diretório `@theme`, e o *tree-shaking* é automático: o plugin `@tailwindcss/vite` detecta quais classes são usadas no código-fonte e gera apenas o CSS necessário, resultando em bundles tipicamente inferiores a 10 KB. A consistência visual foi garantida pela definição de um tema personalizado com cores, espaçamentos e tipografia alinhados à identidade visual do sistema.

2.1.4 shadcn/ui

shadcn/ui é uma coleção de componentes reutilizáveis construídos sobre *Radix UI* (primitivas acessíveis e sem estilo) e estilizados com Tailwind CSS [8]. Diferentemente de bibliotecas tradicionais como Material UI ou Chakra UI, o shadcn/ui não é publicado como um pacote npm instalável — os componentes são copiados diretamente para o código-fonte do projeto, concedendo controle total sobre a implementação e personalização.

No sistema, foram utilizados 46 componentes do shadcn/ui, incluindo `Button`, `Card`, `Dialog`, `Form`, `Table`, `Sidebar`, `Chart` e `Toast`. O utilitário `class-variance-authority` gerencia variantes de componentes, enquanto `tailwind-merge` e `clsx` resolvem conflitos de classes CSS. O `Button`, por exemplo, oferece as variantes `default`, `destructive`, `outline`, `secondary`, `ghost` e `link`, todas definidas via `cva()` e utilizadas consistentemente em toda a interface.

2.1.5 TanStack Router

O TanStack Router é uma biblioteca de roteamento para React que adota uma abordagem declarativa e baseada em arquivos para definição de rotas. Seu diferencial em relação ao React Router DOM tradicional está no suporte a *search params* tipados, *loaders* e *actions* que integram o carregamento de dados ao ciclo de vida da navegação.

No sistema, as rotas foram organizadas em uma árvore aninhada que reflete a hierarquia de navegação do usuário: rotas públicas (login, validação de certificados), rotas protegidas (dashboard, gestão de eventos) e rotas administrativas (configurações, relatórios). O recurso de *beforeLoad* foi utilizado para verificar a autenticação antes de renderizar qualquer rota protegida, redirecionando usuários não autenticados para a tela de login sem necessidade de estado global.

O React Router DOM, embora mais difundido, foi preterido por não oferecer tipagem nativa para *search params* e *path params* — o desenvolvedor precisa declarar tipos manualmente ou usar bibliotecas auxiliares como `zod-router`. O TanStack Router infere os tipos automaticamente a partir da definição da rota, eliminando uma fonte comum de erros em tempo de execução. Além disso, seu modelo de *loaders* e *actions* elimina a necessidade de `useEffect` para buscar dados durante a navegação, substituindo-o por funções assíncronas executadas antes da renderização da rota.

2.1.6 TanStack React Query

O TanStack React Query (anteriormente React Query) é uma biblioteca para gerenciamento de estado assíncrono que abstrai o ciclo de vida completo de requisições a servidores: *cache*, *background refetching*, *paginação*, *optimistic updates* e *deduplicação* de requisições simultâneas [9].

No sistema, o `QueryClient` é instanciado e injetado no contexto do *TanStack Router* (arquivo `router.tsx`), ficando acessível nas funções *loader* das rotas para buscar dados antes da renderização da página. A motivação para sua adoção foi substituir a abordagem manual com `useEffect + useState`, que resulta em código boilerplate e carece de estratégia de cache consistente. Alternativas como Redux Toolkit Query e SWR foram avaliadas, optando-se pelo React Query por sua API mais expressiva para mutações complexas e integração nativa com o ecossistema TanStack.

2.1.7 React Hook Form

React Hook Form é uma biblioteca para gerenciamento de formulários em React que minimiza o número de re-renderizações ao utilizar refs em vez de estado controlado [10]. Diferentemente do Formik, que mantém o estado do formulário no React state e dispara uma re-renderização a cada tecla digitada, o React Hook Form registra os campos via refs e só re-renderiza componentes específicos quando necessário — o que resulta em melhor desempenho em formulários com muitos campos. Sua integração com o Zod via `@hookform/resolvers` permite que os esquemas de validação definidos com Zod sejam reutilizados como regras de formulário, garantindo que as mesmas validações aplicadas no backend sejam executadas no frontend antes do envio.

No sistema, o padrão adotado combina `useForm` com `zodResolver` para cada formulário (emissão de certificado, cadastro de template, etc.). As mensagens de erro retornadas pelo Zod são mapeadas automaticamente para os campos correspondentes e exibidas via componentes `FormMessage` do `shadcn/ui`. O modo `onBlur` dispara a validação no momento em que o usuário sai do campo, oferecendo feedback imediato sem a latência da validação a cada tecla.

2.1.8 Sonner

Sonner é uma biblioteca leve de notificações *toast* para React, desenvolvida por Emil Kowalski [11]. Diferentemente de alternativas como `react-hot-toast` ou `notistack`, o Sonner prioriza a

simplicidade de API (exporta apenas as funções `toast` e `Toaster`) e a acessibilidade, com suporte nativo a leitores de tela.

No sistema, o Sonner é utilizado para exibir feedback ao usuário após operações assíncronas: sucesso na emissão de certificados, erros de validação, confirmação de exclusão e falhas de rede. As notificações são posicionadas no canto superior direito e configuradas para desaparecer automaticamente após 4 segundos, exceto erros que requerem ação manual.

2.1.9 Recharts

Recharts é uma biblioteca de gráficos para React construída sobre os componentes SVG do D3, seguindo uma abordagem declarativa e composível [12]. Cada tipo de gráfico é um componente React independente (`BarChart`, `PieChart`, `LineChart`), facilitando a personalização sem conhecimento aprofundado de SVG.

A escolha do Recharts em vez do `react-chartjs-2` (wrapper para `Chart.js`) deu-se pela integração mais natural com o ecossistema React: enquanto o `Chart.js` manipula o canvas diretamente e exige imperativamente chamar `update()` para refletir alterações nos dados, o Recharts utiliza SVG declarativo e reage automaticamente a mudanças de props — o que simplifica a manutenção de gráficos dinâmicos que se atualizam conforme o usuário filtra os dados no dashboard. O D3 puro foi descartado por sua API de baixo nível, que demandaria mais código boilerplate para alcançar o mesmo resultado.

No sistema, o Recharts é utilizado no dashboard para exibir métricas como total de certificados emitidos por mês (gráfico de barras) e distribuição por tipo de template (gráfico de pizza). Os componentes `ResponsiveContainer`, `Tooltip` e `Legend` garantem que os gráficos se adaptem ao tamanho da tela e exibam informações contextuais ao passar o mouse.

2.1.10 date-fns

`date-fns` é uma biblioteca de utilitários para manipulação de datas em JavaScript, caracterizada por sua abordagem modular e funcional [13]. Cada função é importada individualmente (`format`, `parse`, `differenceInDays`, `isAfter`), evitando o aumento desnecessário do bundle com código não utilizado.

Alternativas como `Moment.js` foram descartadas porque o `Moment.js` é considerado obsoleto pela própria equipe de mantenedores (modo de manutenção desde 2020), possui API mutável que pode causar efeitos colaterais inesperados e não suporta *tree-shaking* — importar

uma única função inclui a biblioteca inteira no bundle final. O `dayjs`, embora mais leve que o `Moment.js`, foi preterido por sua API menos expressiva para operações como *locale* customizado e formatação avançada. O `date-fns`, por sua vez, adota funções puras (sem mutação), é completamente modular e oferece suporte a 80+ locais, incluindo português brasileiro.

No sistema, o `date-fns` é empregado para formatar datas de validade de certificados nos componentes de listagem, calcular diferenças entre datas para alertas de expiração e padronizar a exibição de datas no formato local (`dd/MM/yyyy`) via função `format`. A função `isAfter` é utilizada no modelo `certificate.ts` para determinar se um certificado está expirado.

2.1.11 Vite

Vite é uma ferramenta de build e servidor de desenvolvimento que utiliza módulos ES nativos durante o desenvolvimento para oferecer *Hot Module Replacement* (HMR) instantâneo, eliminando a necessidade de empacotamento em tempo de desenvolvimento [14]. Em produção, utiliza Rollup para gerar bundles otimizados com *code splitting* automático e *lazy loading*.

No projeto, o Vite é a ferramenta de build ****exclusiva do frontend SPA**** — o backend Next.js utiliza seu próprio sistema de compilação (`next build`). Essa divisão reforça a independência entre as duas aplicações: alterações no frontend não exigem reconstrução do backend, e vice-versa. A substituição do Create React App (CRA) pelo Vite reduziu o tempo de inicialização do servidor de desenvolvimento de aproximadamente 30 segundos para menos de 2 segundos, e o HMR reflete alterações no código em milissegundos, melhorando significativamente a produtividade durante o desenvolvimento.

2.2 Integração com a API REST

O frontend comunica-se com um backend externo por meio de uma API REST, cuja URL base é configurada via variável de ambiente `VITE_API_URL`. As requisições são centralizadas em um módulo `api.ts` que utiliza a `fetch` API nativa do JavaScript, encapsulando o tratamento de erros e a serialização JSON. Os endpoints expostos incluem operações CRUD para templates e certificados, emissão de certificados, geração de PDF e verificação de autenticidade.

2.2.1 Zod

Zod é uma biblioteca de validação de esquemas com suporte primário a TypeScript, que permite definir esquemas de dados com inferência automática de tipos [15]. Diferentemente de

alternativas como Joi ou Yup, o Zod foi projetado especificamente para integração com TypeScript, eliminando a necessidade de declarar tipos manualmente: o tipo é inferido diretamente do esquema de validação.

No sistema, os esquemas Zod são utilizados em conjunto com o React Hook Form para validar as entradas dos formulários antes do envio à API. Um mesmo esquema pode validar desde o formato de e-mail até a estrutura completa de um cadastro de certificado, com mensagens de erro localizadas exibidas nos componentes `FormMessage` do `shadcn/ui`. Essa abordagem garante consistência entre a validação no frontend e as regras aplicadas pelo backend.

2.3 Backend

O backend do sistema é uma API REST desenvolvida com Next.js seguindo os princípios da *Clean Architecture*, que separa o código em três camadas: domínio (*entities* e *value objects* sem dependências externas), aplicação (*use cases* e *DTOs*) e infraestrutura (*repositories*, *services* e acesso a banco de dados). As dependências apontam para dentro: a camada de domínio define interfaces que a infraestrutura implementa, permitindo substituir implementações sem alterar a lógica de negócio.

2.3.1 Next.js 16

Next.js é um framework React que, em sua versão 16, oferece o *App Router* com *Route Handlers* para construção de APIs serverless [16]. Cada endpoint é definido como um arquivo em `src/app/api/` exportando funções assíncronas nomeadas (GET, POST, PUT, DELETE) — não há necessidade de um servidor Express ou configuração de rotas manual.

É importante destacar que, neste projeto, o Next.js é utilizado ****exclusivamente** como servidor de API (*backend-only*)*: não há páginas, layouts, *Server Components* ou qualquer funcionalidade de renderização frontend. O diretório `src/app/` contém apenas subdiretórios `api/*` com *Route Handlers*; não existe um `page.tsx` ou `layout.tsx` no projeto. O frontend é uma SPA completamente separada (descrita na Seção 2.1), que consome a API via HTTP. Essa separação foi intencional: o frontend SPA poderia ser substituído por outro cliente (como um aplicativo mobile) sem qualquer alteração no backend, e o backend poderia ser reescrito em outra linguagem sem afetar o frontend. O fato de o Next.js ser tipicamente associado a aplicações full-stack não invalida seu uso como servidor de API puro — os *Route Handlers* operam de forma independente do sistema de páginas e podem substituir frameworks como Express ou

Fastify com menos configuração.

No sistema, os *Route Handlers* do Next.js atuam como a camada mais externa da aplicação: recebem a requisição, validam os parâmetros com Zod, instanciam os *use cases* com as dependências necessárias (*repositories* e *services*) e retornam a resposta. O arquivo `next.config.ts` configura cabeçalhos CORS para permitir requisições do frontend (`http://10.17.40.207:5173`) e isola o PDFKit como pacote exclusivo do servidor via `serverExternalPackages`.

Alternativas como Express e Fastify foram consideradas, mas descartadas porque exigiriam configuração manual de middlewares, roteamento e integração com TypeScript. O Next.js oferece essas funcionalidades de forma nativa (roteamento baseado em arquivos, suporte a TypeScript sem *tsconfig* adicional, variáveis de ambiente validadas com Zod via `next.config.ts`), além de simplificar o deploy ao produzir um único artefato *serverless*. Já o NestJS, embora traga uma arquitetura modular madura, foi preterido por sua curva de aprendizado mais acentuada e pela sobrecarga de abstrações (decorators, módulos, injetores) que não se alinhavam à simplicidade almejada para o projeto.

2.3.2 Supabase e PostgreSQL

Supabase é uma plataforma *open-source* que oferece PostgreSQL como serviço gerenciado, com APIs REST auto-geradas e autenticação integrada [17]. Diferentemente do Firebase (Google), que utiliza um banco NoSQL proprietário, o Supabase utiliza PostgreSQL, um banco relacional maduro que permite consultas complexas, integridade referencial e transações ACID.

A escolha do Supabase em vez de alternativas como Firebase foi motivada pelo modelo de dados relacional do sistema: certificados e templates possuem relações bem definidas (um template pode gerar muitos certificados) e exigem integridade referencial via chaves estrangeiras, algo que o Firestore (NoSQL do Firebase) não oferece nativamente, exigindo validações manuais no código da aplicação. O Appwrite, outra alternativa *open-source* BaaS, foi descartado por seu ecossistema menos maduro e pela ausência de um cliente oficial com suporte a SSR para Next.js no momento da decisão.

No sistema, o Supabase é utilizado exclusivamente como banco de dados relacional. As consultas são feitas via `@supabase/supabase-js` e `@supabase/ssr` (para compatibilidade com server-side rendering do Next.js), sem a utilização de ORMs como Prisma ou TypeORM. A opção por queries SQL diretas — em vez de um ORM — deu-se pelo controle fino sobre as consultas: operações como inserção em lote e CTEs (*Common Table Expressions*) são expres-

sas de forma mais natural em SQL puro, sem as abstrações que um ORM impõe. O esquema do banco de dados contém duas tabelas principais: `templates` (com coluna `layout_config` do tipo JSONB para configuração visual dos certificados) e `certificates` (com índices em `recipient_cpf` e `verification_code` para consultas eficientes de verificação). A nomenclatura segue *snake_case* no banco e *camelCase* no código, com mapeamento manual nos repositórios.

2.3.3 PDFKit

PDFKit é uma biblioteca Node.js para geração de documentos PDF no lado do servidor, sem dependência de ferramentas externas como wkhtmltopdf ou LaTeX [18]. Suporta texto com fontes embutidas, imagens, vetores e cores personalizadas.

A escolha do PDFKit em detrimento de alternativas como Puppeteer baseou-se em eficiência de recursos: o Puppeteer exige a execução de um navegador Chromium headless completo para rasterizar HTML em PDF, consumindo tipicamente mais de 100 MB de memória por instância. Em um ambiente serverless, onde cada requisição pode inicializar uma nova instância, esse custo inviabiliza o uso. O PDFKit opera exclusivamente em Node.js, com footprint de memória da ordem de dezenas de megabytes, e permite controle programático preciso sobre cada elemento do documento — posicionamento absoluto, fontes embutidas e cores arbitrárias — sem a intermediação de um motor de renderização HTML. Bibliotecas como jsPDF foram descartadas por operarem no lado do cliente (navegador), o que as tornaria inadequadas para um fluxo de geração server-side.

No sistema, o `PDFKitService` implementa a interface `IPDFService` definida no domínio, seguindo o princípio de inversão de dependência. O certificado é gerado em formato paisagem A4, com fundo colorido, bordas decorativas, título “CERTIFICADO”, nome do destinatário, curso, carga horária, datas de emissão e validade, CPF formatado e código de verificação. Todas as cores, tamanhos e visibilidade dos elementos são controlados pelo `layoutConfig` do template, armazenado como JSONB no banco de dados. O PDF é retornado como `Buffer` e streamado diretamente na resposta HTTP.

2.3.4 Zod

Zod é uma biblioteca de validação de esquemas que, na versão 4, oferece inferência automática de tipos e composição de esquemas [15]. No backend, os esquemas são definidos em

`application/dtos/index.ts` e utilizados nos *Route Handlers* para validar o corpo das requisições antes de repassá-los aos *use cases*. Essa validação centralizada garante que apenas dados consistentes cheguem à camada de domínio.

2.3.5 Arquitetura e Padrões

A *Clean Architecture* adotada no backend organiza o código em quatro pacotes principais: `domain` (entidades `Certificate` e `Template`, *value object* `VerificationCode`, interfaces de repositório e serviço), `application` (*use cases* como `EmitCertificateUseCase`, `GeneratePDFUseCase`, `VerifyCertificateUseCase` e DTOs de validação), `infrastructure` (implementações concretas `SupabaseCertificateRepository`, `SupabaseTemplateRepository`, `PDFKitService`) e `app` (*Route Handlers* do Next.js). Cada *use case* possui uma única responsabilidade, e as entidades de domínio encapsulam suas próprias regras de negócio (como `Certificate.updateValidity()` que valida a data antes de alterá-la).

2.4 Infraestrutura e Ferramentas

2.4.1 Render

Render é uma plataforma de cloud computing que oferece hospedagem simplificada para aplicações web, sites estáticos e serviços, com deploys automáticos a partir de repositórios Git e certificados SSL gratuitos [19].

O frontend, construído como uma SPA com Vite, é compilado para arquivos estáticos (HTML, CSS, JS) e implantado como um *Static Site* no Render. O deploy é acionado automaticamente a cada *push* na branch principal do GitHub. O arquivo `index.html` na raiz do projeto serve como ponto de entrada, e o roteamento no lado do cliente é gerenciado pelo TanStack Router, que utiliza *History API* para navegação sem recarregamento de página.

Alternativas como Vercel e Netlify foram avaliadas. O Render foi escolhido por dois motivos principais: (1) suporte nativo a sites estáticos com deploys automáticos a partir do GitHub, sem limite de largura de banda na camada gratuita, diferentemente da Vercel que impõe limites de banda em planos gratuitos; e (2) a possibilidade de, futuramente, hospedar o backend como *Web Service* na mesma plataforma, centralizando a gestão de infraestrutura em um único provedor. O Cloudflare Pages também foi descartado por seu ecossistema de *Workers* focado em funções *edge*, modelo que não se aplica a um backend Node.js tradicional com dependências

como PDFKit.

2.4.2 Jest

Jest é um framework de testes JavaScript desenvolvido pelo Facebook, que oferece um conjunto completo de funcionalidades: *runner* de testes, asserções, *mocking*, *code coverage* e *snapshot testing* [20]. Suporte a TypeScript é provido via `ts-jest`.

O Jest foi escolhido em detrimento do Vitest por dois motivos: (1) maturidade e estabilidade — o Jest possui mais de uma década de desenvolvimento, documentação extensa e uma comunidade consolidada, enquanto o Vitest, embora mais rápido por aproveitar o Vite, ainda estava em estágio inicial de adoção no momento da decisão do projeto; e (2) compatibilidade com o ecossistema — diversas bibliotecas do projeto (`@testing-library/react`, `jest-dom`, `ts-jest`) possuem integração madura e testada com o Jest, reduzindo o risco de incompatibilidades durante o desenvolvimento.

No sistema, o Jest está configurado com cobertura mínima de 75%, validada através da flag `-coverage` no pipeline de CI. Testes unitários foram escritos para as *ViewModels* e funções utilitárias, enquanto a configuração de *mocking* permite isolar as chamadas de API durante os testes sem depender de um servidor backend ativo.

2.4.3 ESLint e Prettier

ESLint é uma ferramenta de análise estática de código que identifica padrões problemáticos, erros de sintaxe e violações de boas práticas [21]. Prettier é um formatador de código opinativo que garante consistência estilística em todo o projeto, eliminando discussões sobre espaçamento, aspas e ponto-e-vírgula.

No sistema, ambos foram configurados com plugins específicos: `eslint-plugin-react-hooks` para garantir a correta utilização dos hooks do React, `@typescript-eslint` para regras de tipagem, e `eslint-config-prettier` para evitar conflitos entre as regras do ESLint e a formatação do Prettier. A integração com o VS Code via extensões permite que erros sejam destacados em tempo real e que o código seja formatado automaticamente ao salvar.

2.4.4 Git e GitHub

Git é um sistema de controle de versão distribuído que permite rastrear alterações no código-fonte, gerenciar ramos de desenvolvimento paralelo e colaborar entre múltiplos desenvolvedo-

res. GitHub é a plataforma de hospedagem de repositórios que estende o Git com *Pull Requests*, *Issues*, *Actions* para CI/CD e revisão de código.

O fluxo de trabalho adotado foi o *Git Flow* simplificado, com as branches `main` (produção), `dev` (desenvolvimento) e `feature/*` (funcionalidades específicas). Cada funcionalidade é desenvolvida em uma branch separada e integrada à `dev` via *Pull Request*, que passa por revisão de código antes do merge.

2.4.5 Visual Studio Code

Visual Studio Code é um editor de código-fonte desenvolvido pela Microsoft, baseado no Electron e no protocolo *Language Server Protocol* (LSP), que oferece suporte avançado a TypeScript, depuração integrada, terminal embutido e um ecossistema rico de extensões.

A produtividade foi ampliada com extensões como ESLint (análise estática), Prettier (formatação automática), Tailwind CSS IntelliSense (autocompletar para classes) e GitHub Copilot (sugestões contextuais).

2.5 Síntese das Tecnologias

A Tabela 1 apresenta um resumo das tecnologias discutidas neste capítulo, com suas respectivas finalidades no contexto do sistema.

Tabela 1 – Tecnologias utilizadas no desenvolvimento do sistema

Tecnologia	Versão	Finalidade
Frontend		
React	19	Construção da interface com componentes reutilizáveis
TypeScript	5.x	Tipagem estática e segurança em tempo de compilação
Tailwind CSS	4.x	Estilização utilitária com configuração CSS-first
shadcn/ui	~2.x	Componentes reutilizáveis baseados em Radix UI
TanStack Router	~1.x	Roteamento declarativo com proteção de rotas
TanStack React Query	~5.x	Gerenciamento de cache e estado assíncrono
React Hook Form	~7.x	Gerenciamento performático de formulários
Sonner	~2.x	Notificações toast acessíveis
Recharts	~2.x	Gráficos declarativos para dashboard
date-fns	~4.x	Utilitários de manipulação de datas
Vite	~7.x	Servidor de desenvolvimento e empacotamento
Zod	~3.x / 4.x	Validação de esquemas no frontend (com React Hook Form) e backend
Backend		
Next.js	16	Framework para API REST com Route Handlers
Node.js	22 LTS	Ambiente de execução do servidor
Supabase	~2.x	Banco de dados PostgreSQL gerenciado
PostgreSQL	16	Banco de dados relacional com integridade referencial
PDFKit	~0.18	Geração server-side de certificados em PDF
UUID	~14.x	Geração de identificadores únicos universais
Infraestrutura e Ferramentas		
Jest	~30.x	Testes unitários com cobertura mínima de 70%
ESLint	~9.x	Análise estática e linting de código
Git	~2.x	Controle de versão distribuído
GitHub	SaaS	Repositório remoto e colaboração via Pull Requests
Visual Studio Code	~1.x	Ambiente de desenvolvimento integrado

3.0 Modelagem do Projeto

3.1 Levantamento de Requisitos

Requisitos Funcionais:

3.1.1 Módulo de Autenticação e Controle de Acesso

- RF001 – Permitir login por e-mail e senha.
- RF002 – Permitir redefinição de senha via e-mail.
- RF003 – Validar credenciais antes de conceder acesso.
- RF004 – Permitir cadastro de novos usuários (apenas administrador).
- RF005 – Permitir gerenciamento de perfis e permissões.
- RF006 – Impedir acesso a áreas restritas sem autenticação.
- RF007 – Impedir acesso simultâneo com a mesma conta.

3.1.2 Módulo de Templates de Certificados

- RF008 – Permitir upload de templates (PPTX, DOCX, PDF).
- RF009 – Permitir personalização de templates.
- RF010 – Permitir selecionar template da galeria interna.
- RF011 – Identificar campos dinâmicos automaticamente.
- RF012 – Permitir criação/edição manual de campos dinâmicos (ex: {{nome}}).
- RF013 – Salvar configurações de mapeamento de campos.
- RF014 – Permitir duplicar templates existentes.

3.1.3 Módulo de Entrada de Dados

- RF015 – Permitir cadastro individual de participantes.
- RF016 – Permitir emissão manual para participante único.
- RF017 – Importar dados via CSV.
- RF018 – Importar dados via Excel (.xlsx).
- RF019 – Configurar gatilhos de emissão automática.
- RF020 – Permitir upload de lista em lote.
- RF021 – Processar múltiplos certificados em fila.
- RF022 – Notificar ao término do processamento.

3.1.4 Módulo de Geração de Certificados

- RF023 – Gerar certificados em PDF.
- RF024 – Preencher automaticamente campos dinâmicos.
- RF025 – Gerar código único por certificado.
- RF026 – Registrar data, hora e localidade da emissão.
- RF027 – Permitir reemissão mantendo histórico.
- RF028 – Permitir download individual.
- RF029 – Permitir download em lote.

3.1.5 Módulo de Envio e Comunicação

- RF030 – Permitir personalização do corpo do e-mail com variáveis.
- RF031 – Registrar status de envio (pendente, enviado, falha).
- RF032 – Reenviar automaticamente em caso de falha (até 3 tentativas a cada 10 min).

3.1.6 Módulo de Validação Pública e Antifraude

- RF033 – Disponibilizar portal público de validação.
- RF034 – Validar certificado via código único.
- RF035 – Gerar QR Code no certificado.
- RF036 – Permitir personalização de cores do QR Code.
- RF037 – Exibir informações básicas ao validar (nome, curso, data, validade, autenticidade).

3.1.7 Módulo de Assinaturas Digitais

- RF038 – Permitir adicionar assinatura digitalizada ao template.
- RF039 – Registrar hash de verificação da assinatura.

3.1.8 Módulo de Dashboard e Gestão

- RF040 – Exibir lista completa de certificados emitidos.
- RF041 – Permitir filtros por curso, período, status e instrutor.
- RF042 – Exibir estatísticas mensais de emissão.
- RF043 – Exibir logs de ações realizadas.

3.1.9 Módulo de Auditoria e Segurança Operacional

- RF044 – Registrar responsável por cada certificado gerado.
- RF045 – Registrar data, hora e local de ações relevantes.
- RF046 – Impedir exclusão definitiva de certificados (somente arquivamento).
- RF047 – Restringir horário de emissão por perfil de colaborador.

3.1.10 Módulo de Gestão de Cursos/Eventos

- RF048 – Permitir cadastro de cursos/eventos.
- RF049 – Associar participantes ao curso.
- RF050 – Definir carga horária.
- RF051 – Registrar instrutor responsável.
- RF052 – Configurar datas de início e término.
- RF053 – Listar certificados vinculados ao curso.
- RF054 – Impedir emissão para participante não vinculado.
- RF055 – Registrar local do curso.

3.1.11 Módulo de Participantes

- RF056 – Cadastro completo de participantes (nome, e-mail, CPF opcional).
- RF057 – Permitir edição de dados.
- RF058 – Importar participantes sem duplicidade.
- RF059 – Exibir histórico de certificados por participante.
- RF060 – Permitir busca por nome, e-mail e CPF.

3.1.12 Módulo Multiempresa / Multi-tenant

- RF061 – Permitir cadastro de múltiplas organizações.
- RF062 – Isolar dados entre organizações.
- RF063 – Permitir que cada empresa tenha seus próprios templates.
- RF064 – Permitir que administradores gerenciem usuários internos.

3.1.13 Módulo de Configurações do Sistema

- RF065 – Personalizar mensagens padrão.
- RF066 – Ativar/desativar envio automático.
- RF067 – Configurar regras de reemissão.

3.1.14 Módulo de Armazenamento e Arquivos

- RF068 – Permitir exclusão lógica (soft-delete) de arquivos.
- RF069 – Armazenar templates no sistema.

3.1.15 Módulo de Notificações e Alertas

- RF070 – Notificar falhas de emissão.
- RF071 – Notificar sucesso no processamento de planilhas.
- RF072 – Alertar quando template estiver incompleto ou inválido.

3.1.16 Módulo de Monitoramento e Logs

- RF073 – Registrar logs de autenticação e tentativas de acesso.
- RF074 – Registrar eventos críticos (falha geração PDF etc.).
- RF075 – Permitir exportação de logs.
- RF076 – Registrar alterações em templates e configurações (controle de versão).

3.1.17 Módulo de Suporte, Operação e Manutenção

- RF077 – Permitir abertura de chamados de suporte.
- RF078 – Exibir status do sistema (online/manutenção).
- RF079 – Permitir atualização de termos de uso e políticas.

Requisitos Não Funcionais:

3.1.18 Segurança

- RNF001 – O sistema deve utilizar o Supabase Auth para gerenciamento seguro de autenticação e armazenamento de senhas com hash criptográfico seguro (bcrypt).
- RNF002 – O sistema deve utilizar conexão segura HTTPS (TLS 1.2 ou superior).
- RNF003 – O sistema deve proteger dados sensíveis em repouso utilizando criptografia fornecida pela infraestrutura cloud.
- RNF004 – O sistema deve bloquear autenticação após 5 tentativas consecutivas inválidas (conforme política do provedor de autenticação).
- RNF005 – Sessões inativas devem expirar automaticamente após tempo configurável (ex.: 30 minutos).
- RNF006 – O sistema deve implementar controle de acesso baseado em papéis (RBAC).
- RNF007 – O sistema deve seguir boas práticas de segurança conforme OWASP Top 10.
- RNF008 – O sistema deve registrar logs de autenticação e eventos de segurança.
- RNF009 – O sistema deve garantir isolamento de dados entre empresas (multi-tenant).
- RNF010 – O sistema deve permitir futura implementação de autenticação multifator (MFA).

3.1.19 Desempenho e Escalabilidade

- RNF011 – O sistema deve suportar geração simultânea mínima de 500 certificados por lote.
- RNF012 – O tempo médio de geração individual não deve exceder 5 segundos.
- RNF013 – O portal público de validação deve responder em até 2 segundos.
- RNF014 – O sistema deve permitir escalabilidade horizontal do serviço de geração.
- RNF015 – O sistema deve suportar volume mínimo de 10.000 certificados/mês.
- RNF016 – A geração em lote deve ser processada de forma assíncrona por meio de fila de processamento, sem bloquear a aplicação principal.

- RNF017 – A fila de certificados deve garantir processamento ordenado e controle de concorrência.

3.1.20 Disponibilidade e Confiabilidade

- RNF018 – O sistema deve garantir disponibilidade mínima de 99,5% mensal.
- RNF019 – O sistema deve realizar backup automático diário do banco de dados.
- RNF020 – O sistema deve permitir restauração de dados em até 24 horas.
- RNF021 – O sistema deve implementar mecanismo de retentativa automática para tarefas falhas na fila de certificados.
- RNF022 – O sistema deve registrar falhas de processamento para auditoria posterior.

3.1.21 Usabilidade e Acessibilidade

- RNF023 – O sistema deve ser responsivo (desktop, tablet e mobile).
- RNF024 – O sistema deve manter padrão visual consistente.
- RNF025 – O sistema deve fornecer mensagens de erro claras.
- RNF026 – O sistema deve permitir navegação por teclado.
- RNF027 – O sistema deve seguir recomendações WCAG 2.1 nível AA (quando aplicável).

3.1.22 Compatibilidade e Integração

- RNF028 – O sistema deve ser compatível com navegadores modernos (Chrome, Firefox, Edge, Safari).
- RNF029 – A geração de PDF deve preservar fidelidade visual do template original.
- RNF030 – O sistema deve disponibilizar API REST autenticada via token JWT do Supabase.
- RNF031 – O sistema deve aceitar arquivos CSV codificados em UTF-8.
- RNF032 – O sistema deve permitir integração via Webhooks em formato JSON.

3.1.23 Armazenamento e Retenção

- RNF033 – O sistema deve armazenar certificados por no mínimo 5 anos.
- RNF034 – O sistema deve garantir integridade dos certificados por meio de hash único.
- RNF035 – O sistema deve aplicar exclusão lógica (soft delete) quando aplicável.

3.1.24 Conformidade Legal e LGPD

- RNF036 – O sistema deve estar em conformidade com a LGPD (Lei nº 13.709/2018).
- RNF037 – O sistema deve permitir exclusão ou anonimização de dados pessoais mediante solicitação.
- RNF038 – O sistema deve registrar consentimento para tratamento de dados.
- RNF039 – Logs sensíveis devem ser acessíveis apenas por administradores autorizados.
- RNF040 – O sistema deve manter trilha de auditoria inviolável para operações críticas.

3.1.25 Arquitetura e Tecnologia

- RNF041 – O sistema deve adotar arquitetura baseada em microserviços, com serviços independentes e desacoplados.
- RNF042 – Cada microserviço deve possuir responsabilidade única e bem definida (ex.: autenticação, geração de certificados, envio de e-mail, validação pública).
- RNF043 – A comunicação entre microserviços deve ocorrer via API REST segura ou mensageria assíncrona.
- RNF044 – O sistema deve utilizar API Gateway como ponto único de entrada para requisições externas.
- RNF045 – A geração em lote deve ser processada por um microserviço específico de processamento assíncrono utilizando fila de mensagens.
- RNF046 – Cada microserviço deve permitir deploy independente sem impactar os demais.

- RNF047 – O sistema deve permitir escalabilidade horizontal individual por serviço.
- RNF048 – O sistema deve utilizar banco de dados relacional com integridade referencial (PostgreSQL).
- RNF049 – O sistema deve definir estratégia de isolamento multi-tenant (por schema ou coluna organizacional).
- RNF050 – O sistema deve implementar monitoramento e observabilidade centralizada (logs, métricas e rastreamento).
- RNF051 – O sistema deve permitir deploy em ambiente cloud compatível com containers (ex.: Docker).

3.1.26 Manutenção e Monitoramento

- RNF052 – O sistema deve permitir atualização com mínima indisponibilidade (deploy contínuo).
- RNF053 – O sistema deve possuir documentação técnica e manual do usuário.
- RNF054 – O sistema deve registrar erros em sistema centralizado de monitoramento.
- RNF055 – O sistema deve permitir extensibilidade sem impacto nas funcionalidades existentes.

3.1.27 Internacionalização

- RNF056 – O sistema deve permitir configuração de timezone por organização.
- RNF057 – O sistema deve permitir tradução futura da interface.
- RNF058 – O sistema deve suportar diferentes formatos de data e hora.

3.4 Arquitetura do Sistema

Arquitetura Backend - Clean Architecture (Visão Geral)

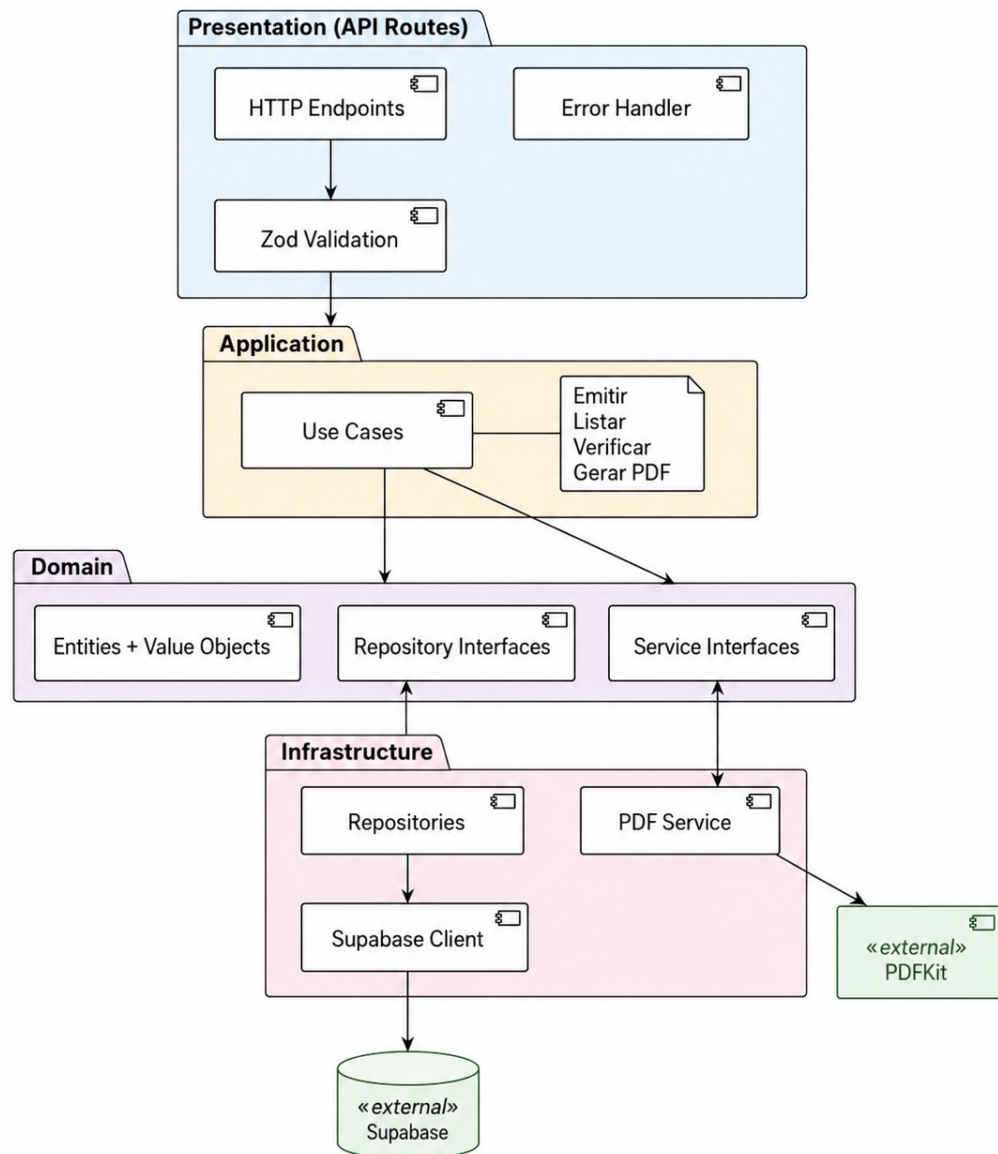


Figura 3 – Arquitetura do Backend - Clean Architecture com Next.js e Supabase

Frontend CertifyHub - Arquitetura MVVM (Visão Geral)

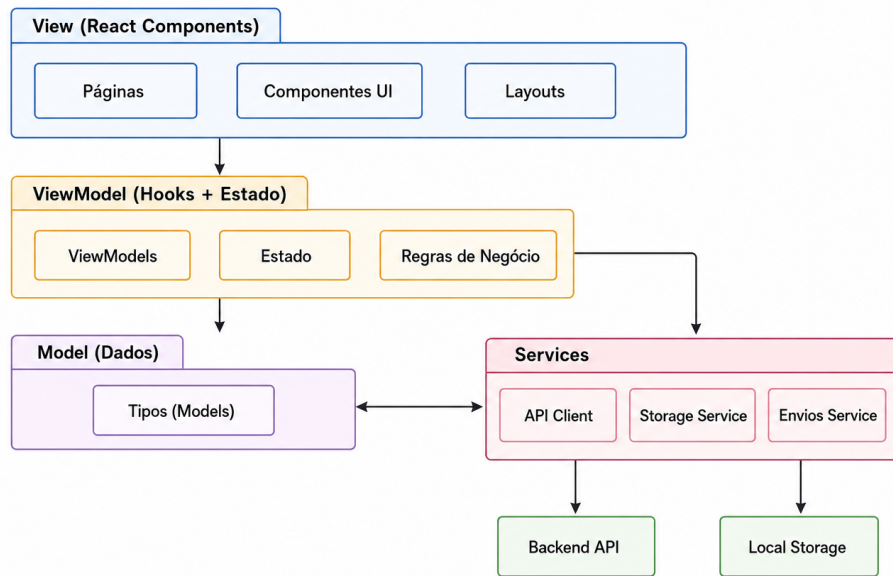


Figura 4 – Arquitetura do Frontend - Padrão MVVM com React e TanStack

3.5 Diagrama Entidade-Relacionamento

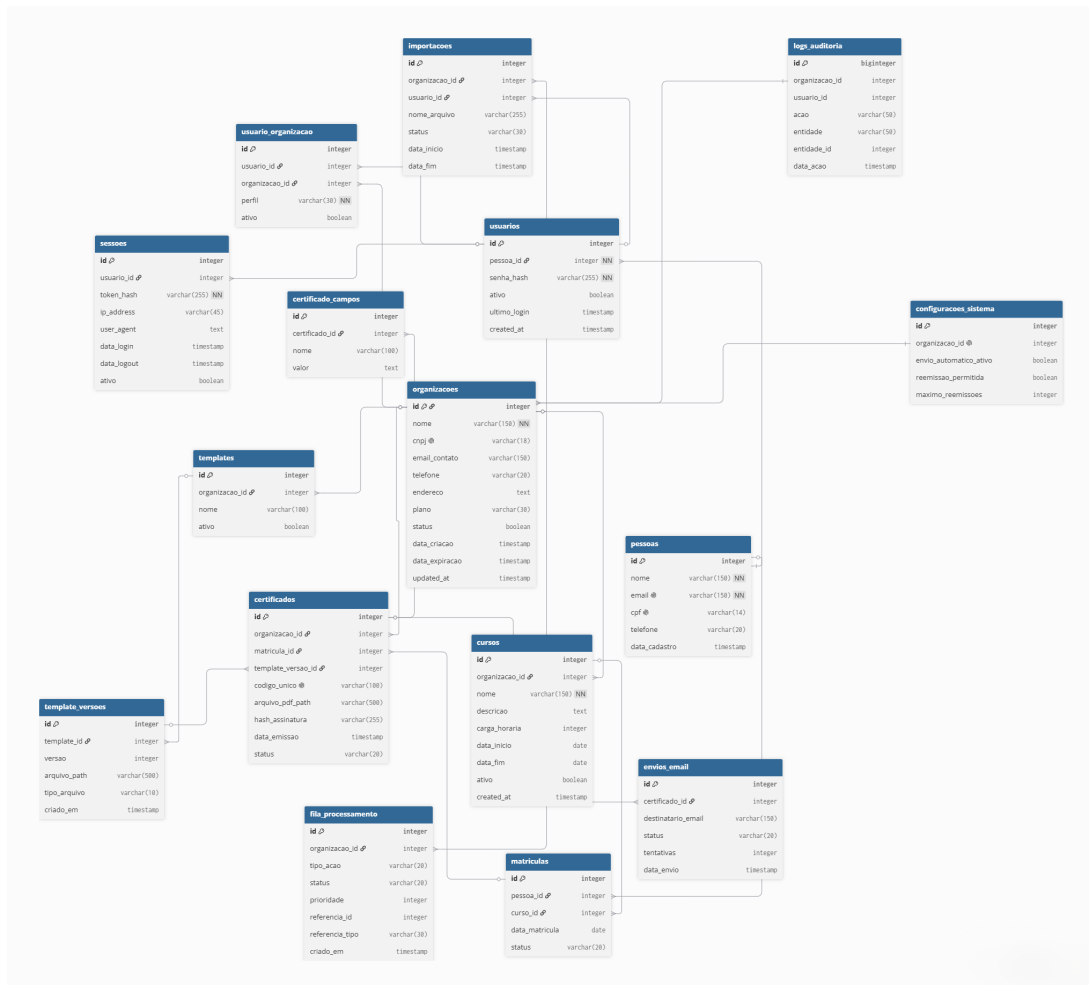


Figura 5 – Diagrama Entidade-Relacionamento (DER) do Banco de Dados

3.6 Interfaces

Inserir prints das telas ou protótipos.

4.0 Software

4.1 Implantação

Instruções de instalação e execução.

4.2 Testes

Tabela de testes.

5.0 Considerações Finais

Síntese, limitações e sugestões.

Referências

- [1] RAHMAN, K. A.; AHMED, S. A. A systematic literature review on qr code–based systems for academic credential verification. *e-Jurnal Penyelidikan dan Inovasi*, v. 13, n. 1, p. 300–315, 2026.
- [2] LORIEN, A.; WELLEM, T. Implementasi sistem otentikasi dokumen berbasis quick response (qr) code dan digital signature. *Jurnal RESTI (Rekayasa Sistem dan Teknologi Informatika)*, v. 5, n. 4, p. 663–671, 2021.
- [3] KULKARNI, R. et al. A secure certificate authentication system using cryptographic hashing and qr code verification. *International Journal of Sciences and Innovation Engineering*, v. 2, n. 10, p. 1407–1414, 2025. Disponível em: <<https://ijsci.com/index.php/home/article/view/840>>.
- [4] OLAMIDE, O. O. et al. Design and implementation of a certificate verification system using quick response (qr) code. *LAUTECH Journal of Computing and Informatics*, v. 2, n. 1, p. 35–40, 2021. Disponível em: <<http://laujci.lautech.edu.ng/index.php/laujci/article/view/36>>.
- [5] KORA, H. B.; MANITA, M. Modern front-end web architecture using react.js and next.js. *University of Zawia Journal of Engineering Sciences and Technology*, v. 2, n. 1, p. 1–13, 2024.
- [6] MICROSOFT. *TypeScript Documentation — TypeScript 5.x*. 2025. Disponível em: <<https://www.typescriptlang.org/docs/>>.
- [7] LABS, T. *Tailwind CSS Documentation*. 2025. Disponível em: <<https://tailwindcss.com/docs>>.
- [8] SHADCN. *shadcn/ui Documentation*. 2025. Disponível em: <<https://ui.shadcn.com/docs>>.
- [9] LINSLEY, T.; CONTRIBUTORS, T. *TanStack Query Documentation*. 2025. Disponível em: <<https://tanstack.com/query/latest>>.
- [10] BLUEBILL1049; CONTRIBUTORS, R. H. F. *React Hook Form Documentation*. 2025. Disponível em: <<https://react-hook-form.com/>>.

- [11] KOWALSKI, E. *Sonner — Toast Component for React*. 2025. Disponível em: <<https://sonner.emilkowal.ski/>>.
- [12] GROUP, R. *Recharts Documentation*. 2025. Disponível em: <<https://recharts.org/en-US/>>.
- [13] CONTRIBUTORS *date-fns. date-fns Documentation*. 2025. Disponível em: <<https://date-fns.org/docs/>>.
- [14] TEAM, V. *Vite Documentation*. 2025. Disponível em: <<https://vitejs.dev/docs/>>.
- [15] HACKS, C. *Zod Documentation*. 2025. Disponível em: <<https://zod.dev/>>.
- [16] VERCEL. *Next.js Documentation*. 2025. Disponível em: <<https://nextjs.org/docs>>.
- [17] SUPABASE. *Supabase Documentation*. 2025. Disponível em: <<https://supabase.com/docs>>.
- [18] FOLIOJS. *PDFKit — A JavaScript PDF Generation Library for Node and the Browser*. 2025. Disponível em: <<https://pdfkit.org/>>.
- [19] RENDER. *Render Documentation*. 2025. Disponível em: <<https://render.com/docs>>.
- [20] FOUNDATION, O. *Jest Documentation*. 2025. Disponível em: <<https://jestjs.io/docs/>>.
- [21] FOUNDATION, O. *ESLint Documentation*. 2025. Disponível em: <<https://eslint.org/docs/latest/>>.

6.0 Anexos

6.1 Declaração de Entrega do Código-Fonte

Declaro que o código-fonte foi entregue [...]

Assinatura: _____

6.2 Declaração de Submissão ao NIT

Declaro que o software foi submetido ao NIT [...]

Assinatura: _____